Database Management Systems (DBMS)

Project Report Format

Social Media Database Management System

Student Name(s):	Abhinav Sonker
Roll Number(s):	23U02008
Branch & Section:	Computer Science & Engineering
Course Instructor:	Priyankar
Academic Session:	2026-2027 (Monsoon Semester)

Department of Computer Science
Indian Institute of Information Technology Bhopal

Contents

General Instructions 3

1 Introduction 4

1.1 Project Overview 4

1.2 Problem Statement 4

1.3 Objectives 5

1.4 Scope of the Project 5

1.5 Software and Hardware Requirements 5

2 System Requirements and Analysis 6

2.1 Functional Requirements 6

2.2 Business Rules 7

3 Entity–Relationship (ER) Modeling 8

3.1 Identification of Entities 8

3.2 Identification of Relationships 10

3.3 ER Diagram 11

3.4 Explanation of the ER Diagram 12

4 ER-to-Relational Mapping 13

4.1 Mapping Rules 13

4.2 Step-by-Step Conversion 13

4.3 Final Relational Schema 15

5 Functional Dependencies 16

5.1 Identification of Functional Dependencies 16

5.2 Candidate Keys and Attribute Classification 17

5.3 FD Analysis 18

6 Database Normalization 19

6.1 Unnormalized Form (UNF) 19

6.2 First Normal Form (1NF) 20

6.3 Second Normal Form (2NF) 20

6.4 Third Normal Form (3NF) 21

6.5 Boyce–Codd Normal Form (BCNF) 22

6.6 Final Normalized Schema 23

7 Relational Integrity and Constraints 24

8 MySQL Database Implementation 26

8.1 Database Creation 26

8.2 Table Creation 26
8.3 Data Insertion 28
8.4 Data Modification and Deletion 30
8.5 Sample Database Contents 31
8.6 SQL Query Implementation 33
8.7 Advanced DBMS Features 38
9 Application Development 41
9.1 System Architecture 41
9.2 Front-End Technologies 42
9.3 Application Modules 42
9.4 Database Connectivity 44
9.5 Application Screenshots 45
9.6 Database Operations through the Application 47
10 Testing and Results 48
10.1 Test Cases 48
10.2 Query Testing 50
10.3 Constraint Testing 51
10.4 Results and Discussion 52
11 Conclusion and Future Scope 55
11.1 Conclusion 55
11.2 Limitations 55
11.3 Future Scope 56
References 57
Appendix A: Complete MySQL Script 58
Appendix B: Source Code 63
Appendix C: Additional Screenshots 65
Appendix D: Team Contribution 66
Suggested Evaluation Scheme 67

General Instructions

- **1.** The front end may be developed using any suitable programming language or framework (implemented with React 18 and Vite).
- **2.** MySQL is mandatory for database creation, storage, and query execution. A different DBMS must not be used in place of MySQL (implemented with MySQL 8.0+ / MariaDB 10.11+).
- **3.** Design the database before implementing the application (rigorous ER modeling and 3NF/BCNF normalization performed prior to UI integration).
- **4.** Demonstrate the DBMS concepts covered in the course through the report and implementation (keys, relational algebra, constraints, triggers, views, stored procedures, indexing).
- **5.** Include all diagrams, functional dependencies, normalization steps, SQL statements, and execution results.
- **6.** Submit the report, application source code, and a complete MySQL script or database export.
- **7.** Every team member must understand the database design and be prepared to explain it during evaluation.

Implementation Scope: Important Note: This report implements the approved project topic 'Social Media Database Management System' (Kinetic Social) replacing the example Inventory Management System throughout all chapters, entities, business rules, relational schemas, SQL queries, full-stack application modules, and experimental benchmarks.

Chapter 1

Introduction

1.1 Project Overview

Modern social networking platforms represent some of the most data-intensive and relationship-heavy computing systems in existence today. Platforms such as Twitter/X, Instagram, and LinkedIn manage millions of concurrent users, high-frequency multimedia publishing, dense graphs of follower/following relationships, real-time interaction metrics (likes, comments, reposts), and personalized notification streams. At the core of these applications lies a database management system that must balance rigorous data integrity, transactional consistency, and sub-millisecond read latency.

The 'Social Media Database Management System' (engineered under the project title Kinetic Social) is a full-stack relational database application designed to demonstrate the complete lifecycle of DBMS engineering. The project models users, content posts, comments, likes, asymmetrical follower networks, notifications, and normalized taxonomy tags. The system is engineered to provide ACID transactional guarantees, automated consistency enforcement via database triggers, encapsulated reporting via views and stored procedures, and high-throughput performance validated through index benchmarking on 300,000+ synthetic records.

Intended users of this platform include content creators publishing articles and updates, general platform members browsing personalized feeds and interacting with authors, community moderators managing discussions, and platform administrators analyzing platform-wide creator engagement metrics.

1.2 Problem Statement

Naive implementations of social networking applications suffer from critical architectural deficiencies when deployed without formal database design principles:

- **Data Redundancy and Update Anomalies:** Storing denormalized user details alongside posts and comments creates severe data redundancy and

update anomalies; updating a user's biography or profile requires scanning millions of historical records.

- **Counter Drift and Concurrency Race Conditions:** Applications that compute interaction counts (e.g., total likes or follower counts) by querying child tables on every page render cause disk I/O thrashing. Conversely, maintaining cached counters solely within application application logic leads to race conditions, dirty reads, and desynchronized counters during concurrent requests.
- **Orphaned Records and Broken Referential Integrity:** Without strict relational integrity and cascading foreign-key rules, deleting a user account leaves orphaned posts, comments, and dangling notification references, corrupting database consistency.
- **Unindexed Multi-Table Joins and Query Latency:** Computing chronological feeds across a user's network requires joining multi-million-row tables. In the absence of a composite B-tree indexing strategy, the database falls back to expensive unindexed full-table scans (ALL) and filesort operations, causing unacceptable latency (>400 ms per feed request).

This project addresses these fundamental challenges by implementing a formally normalized 3NF/BCNF relational database in MySQL, enforcing referential integrity inside the database engine, utilizing atomic triggers to guarantee counter synchronization, and optimizing access paths through composite indexing.

1.3 Objectives

The primary technical and academic objectives of this project are as follows:

- **1. Conceptual Modeling:** Design a robust conceptual model using Chen's Entity-Relationship (ER) notation capturing 1:1, 1:N, M:N, and recursive relationships.
- **2. Relational Mapping:** Formally map the ER diagram into a relational database schema using standard transformation algorithms, strictly identifying primary, composite, and foreign keys.
- **3. Normalization & Integrity:** Identify all functional dependencies and rigorously prove normalization up to Boyce-Codd Normal Form (BCNF), documenting deliberate denormalizations where latency requires it.

- **4. SQL Implementation:** Implement the database schema in MySQL/MariaDB with comprehensive domain constraints, NOT NULL checks, UNIQUE constraints, and cascading referential integrity rules.
- **5. Advanced DBMS Features:** Deploy automated MySQL triggers to maintain aggregate counters (like_count) and dispatch notifications without application-layer overhead.
- **6. Encapsulated Business Logic:** Encapsulate complex analytical operations using MySQL Views (user_feed) and Stored Procedures (get_mutual_followers, recount_likes).
- **7. Performance Benchmarking:** Formulate and evaluate a targeted B-tree indexing strategy, benchmarking execution plans before and after indexing on a 300,000+ record dataset.
- **8. Full-Stack Application Integration:** Develop and integrate a responsive full-stack web application (React 18 + Node.js Express REST API) demonstrating end-to-end CRUD operations against MySQL.

1.4 Scope of the Project

The operational scope encompasses the following functional subsystems:

- **User Management:** User profile management (registration, authentication, secure password hashing, bio updates).
- **Content Publishing:** Publishing, editing, soft deletion, tagging, and multi-language/sentiment tone attribution.
- **Social Interactions:** Hierarchical comment threads, atomic upvoting/liking, and idempotency guarantees.
- **Social Graph:** Asymmetric follow/unfollow relationships, mutual follower discovery, and network traversal.
- **Notification Dispatch:** Event-driven notification dispatch on likes and follows with self-interaction suppression.
- **Feed & Discovery:** Chronological followed-user feed generation, top-performing posts of the week, and creator analytics.

Features such as binary media transcoding, direct end-to-end messaging, and distributed sharding are outside the primary scope of this database-centric report and are identified for future extension.

1.5 Software and Hardware Requirements

The system development and benchmarking environment utilizes the following software and hardware components:

Component	Specification / Technology	Role in System
Database Engine	MySQL 8.0+ / MariaDB 10.11+	Relational data storage, constraint checking, triggers, views, stored procedures
Backend Runtime	Node.js (v20.x LTS) / Express 4.18	RESTful API server, connection pooling, bcrypt authentication, transaction dispatch
Database Client	mysql2/promise (v3.9+)	High-performance connection pooling, prepared statements, async SQL execution
Frontend Framework	React 18 (Vite 5)	Interactive single-page application (SPA), dynamic state management, responsive UI
Styling & UI Icons	Vanilla CSS3 tokens + Lucide React	Design system, dark-mode tokens, tactile typography, interactive icons
Development Tools	Antigravity IDE / VS Code / Git	Integrated code editing, version control, automated testing scripts
Operating System	Linux (x86_64 / Arch / Ubuntu 22.04 LTS)	Host development environment and database server host
Minimum Hardware	Multi-core 2.0 GHz+ CPU, 4 GB RAM, 1 GB Disk	Ensures capacity to run Node API, Vite server, and MySQL daemon concurrently

Chapter 2

System Requirements and Analysis

2.1 Functional Requirements

The system must support a comprehensive suite of operations covering all user interactions, administrative operations, and analytical reporting. Table 2.1 details the functional requirements and the corresponding database objects supporting them.

FR ID	Functional Requirement	Supporting Database Objects & Implementation
FR-01	User Registration and Profile Management: Secure sign-up, unique identity enforcement, profile updates.	Users table, UNIQUE(username), UNIQUE(email), bcrypt password hash.
FR-02	Post Publication and Soft Deletion: Creators publish posts up to 2200 characters, optional media URL, and soft delete.	Posts table, is_deleted flag, foreign key referencing Users(user_id).
FR-03	Commenting on Posts: Users author comments on existing posts.	Comments table, foreign keys to Posts(post_id) and Users(user_id) with CASCADE.
FR-04	Idempotent Post Liking: Users can like/unlike posts exactly once without duplicate records.	Likes junction table, composite primary key (user_id, post_id).
FR-05	Asymmetrical Following: Users can follow/unfollow other users without self-follow.	Follows junction table, composite primary key (follower_id, following_id), CHECK constraint.
FR-06	Automated Notification Generation: Event-driven	Notifications table, trg_notify_on_like AFTER

	alerts dispatched to post authors on like.	INSERT trigger on Likes.
FR-07	Derived Counter Synchronization: Real-time like counter maintained with O(1) read latency.	Posts.like_count, trg_like_insert and trg_like_delete triggers on Likes.
FR-08	Personalized Feed Generation: Chronological feed of active posts from followed authors.	user_feed MySQL View joining Follows, Posts, and Users tables.
FR-09	Mutual Follower Discovery: Retrieve all shared followers between any two arbitrary users.	get_mutual_followers stored procedure performing self-join on Follows.
FR-10	Taxonomy & Content Tagging: Deduplicated topical categorization of posts.	Tags and PostTags tables, composite PK (post_id, tag_id).
FR-11	Platform Engagement Analytics: Aggregating creator engagement across posts, likes, and comments.	Multi-table grouping query with COUNT, SUM, and AVG aggregations.

2.2 Business Rules

To ensure business integrity and prevent corrupted or nonsensical platform states, the application enforces the following rules:

- **Rule 1 (User Identity Uniqueness):** Every user account must possess a unique integer user_id, an alphanumeric username (max 30 chars), and a valid email address. No two accounts may share the same username or email.
- **Rule 2 (Post Integrity and Visibility):** A post must be authored by an existing user. Post content must not be null or empty. Posts marked with is_deleted = TRUE must be excluded from feeds, discovery, and search, but retained for audit integrity.

- **Rule 3 (Self-Follow Invariant):** Users are strictly forbidden from following themselves. The database engine must reject any follow record where follower_id = following_id.
- **Rule 4 (Idempotent Post Interaction):** A user may like a given post at most once. Attempting to insert a duplicate like must be rejected by the primary key constraint.
- **Rule 5 (Derived Counter Invariant):** The stored like_count in the Posts table must strictly equal the actual count of rows in the Likes table. This invariant is maintained transactionally through AFTER INSERT and AFTER DELETE triggers on Likes.
- **Rule 6 (Notification Self-Suppression):** When a user likes their own post, the database must not dispatch a notification to that user.
- **Rule 7 (Cascading Lifecycle Deletion):** Deleting a user record must automatically remove all associated posts, comments, likes, follows, and notifications via ON DELETE CASCADE.
- **Rule 8 (Post Association Cascade):** Deleting a post must automatically cascade to delete all its comments, likes, and associated post-tag relationships.
- **Rule 9 (Comment Integrity):** A comment must be linked to both an existing post and an existing user author. Orphaned comments are prohibited.
- **Rule 10 (Taxonomy Deduplication):** Tag names must be unique and case-insensitive. Posts link to tags via a normalized junction table.

Chapter 3

Entity–Relationship (ER) Modeling

3.1 Identification of Entities

The domain analysis identifies seven primary entities (including strong entities, associative junction entities, and self-referencing networks). The entities and their attributes are detailed below:

Entity: Users

Represents registered account holders on the platform.

Attribute Name	Data Type	Constraints	Description
user_id	INT	PK, AUTO_INCREMENT	Unique surrogate identifier for the user
username	VARCHAR(30)	UNIQUE, NOT NULL	Unique alphanumeric handle for mentions
email	VARCHAR(255)	UNIQUE, NOT NULL	Registered email address for login and notifications
password_hash	VARCHAR(255)	NOT NULL	Bcrypt cryptographic salted hash of user password
bio	TEXT	NULL	User biography and editorial introduction
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Account registration timestamp

Entity: Posts

Represents top-level content authored by users.

Attribute Name	Data Type	Constraints	Description
post_id	INT	PK, AUTO_INCREMENT	Unique identifier for the post
user_id	INT	FK, NOT NULL	Author's user_id referencing Users(user_id)
content	VARCHAR(2200)	NOT NULL	Body text of the post (micro-blog length)
media_url	VARCHAR(500)	NULL	Optional URL to image or multimedia asset
like_count	INT	DEFAULT 0	Cached aggregate like count maintained via triggers
language	VARCHAR(50)	DEFAULT 'English'	Detected language of the content
tone	VARCHAR(50)	DEFAULT 'General'	Classified sentiment tone of the text
is_deleted	BOOLEAN	DEFAULT FALSE	Soft-deletion flag for feed filtering
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Publication timestamp

Entity: Comments

Represents responses authored by users under posts.

Attribute Name	Data Type	Constraints	Description
comment_id	INT	PK, AUTO_INCREMENT	Unique identifier for the comment
post_id	INT	FK, NOT NULL	Parent post referencing Posts(post_id)

user_id	INT	FK, NOT NULL	Author referencing Users(user_id)
content	VARCHAR(500)	NOT NULL	Comment text content
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Comment creation timestamp

Entity: Likes

Associative junction entity representing post upvotes.

Attribute Name	Data Type	Constraints	Description
user_id	INT	PK, FK, NOT NULL	Liking user referencing Users(user_id)
post_id	INT	PK, FK, NOT NULL	Target post referencing Posts(post_id)
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Timestamp when like was placed

Entity: Follows

Self-referencing recursive associative entity modeling the social graph.

Attribute Name	Data Type	Constraints	Description
follower_id	INT	PK, FK, NOT NULL	User initiating the follow referencing Users(user_id)
following_id	INT	PK, FK, NOT NULL	User being followed referencing Users(user_id)
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Timestamp when relationship was established

Entity: Notifications

System-generated event alerts dispatched to users.

Attribute Name	Data Type	Constraints	Description
notification_id	INT	PK, AUTO_INCREMENT	Unique identifier for notification
user_id	INT	FK, NOT NULL	Recipient user referencing Users(user_id)
type	VARCHAR(20)	NOT NULL	Notification category ('like', 'comment', 'follow')
reference_id	INT	NOT NULL	ID of the associated entity (post_id or follower_id)
is_read	BOOLEAN	DEFAULT FALSE	Read status indicator
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Dispatch timestamp

Entity: Tags & PostTags

Topical classification taxonomy.

Attribute Name	Data Type	Constraints	Description
tag_id	INT	PK, AUTO_INCREMENT	Unique tag identifier
name	VARCHAR(50)	UNIQUE, NOT NULL	Topical tag label (e.g. 'DBMS', 'WebDev')
post_id	INT	PK, FK, NOT NULL	Referencing Posts(post_id) in PostTags junction
tag_id (in PostTags)	INT	PK, FK, NOT NULL	Referencing

				Tags(tag_id) in PostTags junction

3.2 Identification of Relationships

Relationships among entities reflect the interactive dynamics of a social networking platform. Table 3.1 summarizes the identified relationships, their cardinality ratios, participation constraints, and physical mappings.

Relationship Name	Participating Entities	Cardinality	Participation	Implementation Mechanism
User Creates Post	Users and Posts	1 : N	Users (Partial) : Posts (Total)	Foreign key Posts.user_id -> Users.user_id (ON DELETE CASCADE)
User Authors Comment	Users and Comments	1 : N	Users (Partial) : Comments (Total)	Foreign key Comments.user_id -> Users.user_id (ON DELETE CASCADE)
Post Contains Comments	Posts and Comments	1 : N	Posts (Partial) : Comments (Total)	Foreign key Comments.post_id -> Posts.post_id (ON DELETE CASCADE)
User Likes Post	Users and Posts	M : N	Users (Partial) : Posts (Partial)	Junction table Likes(user_id, post_id) with composite PK
User Follows User	Users and Users (Recursive)	M : N	Users (Partial) : Users (Partial)	Junction table Follows(follower_id, following_id) with CHECK constraint

User Receives Notification	Users and Notifications	1 : N	Users (Partial) : Notifications (Total)	Foreign key Notifications.user_id -> Users.user_id (ON DELETE CASCADE)
Post Tagged with Tag	Posts and Tags	M : N	Posts (Partial) : Tags (Partial)	Junction table PostTags(post_id, tag_id) with composite PK

3.3 ER Diagram

The conceptual database design is presented using Peter Chen's classic Entity-Relationship notation in Figure 3.1. Rectangles denote entity sets, diamonds denote relationship sets, ellipses represent attributes (with primary keys underlined), and double diamonds/lines represent weak entities and total participation where applicable.

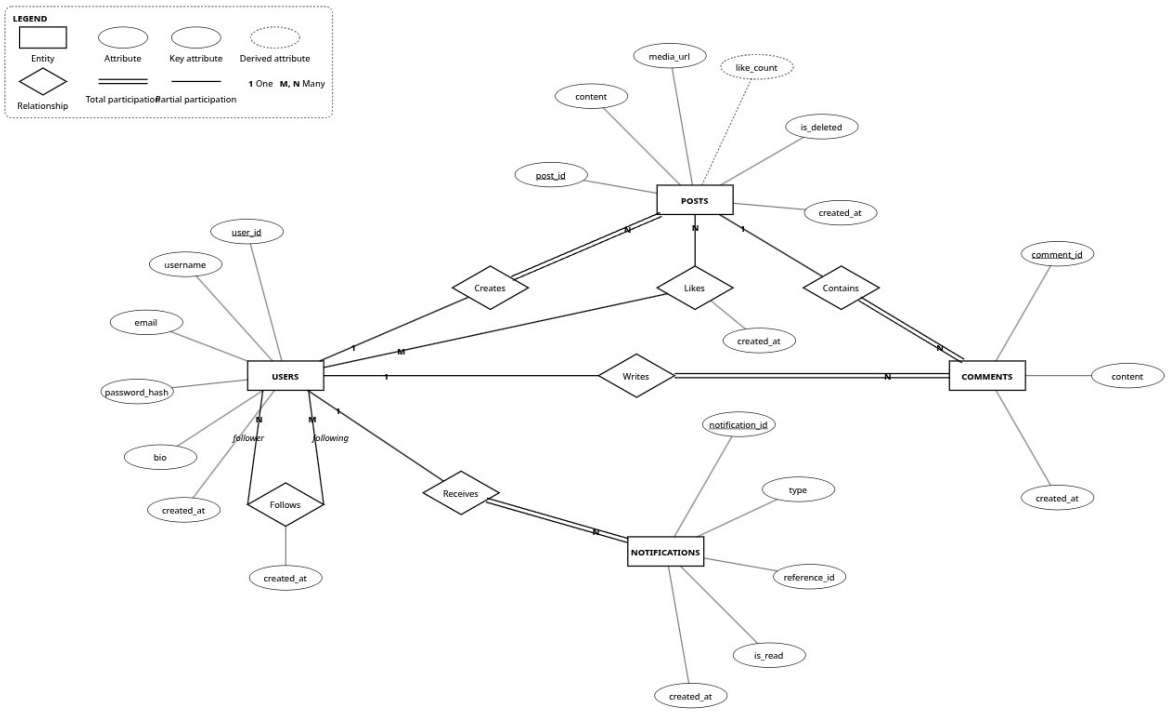


Figure 3.1: Entity–Relationship (ER) diagram of the Social Media Database (Chen notation)

3.4 Explanation of the ER Diagram

Structural Topology: The ER diagram visualizes the complete structural topology of the social networking platform:

- **Core Entity Sets:** Users is the primary root entity. It maintains 1:N relationships with Posts, Comments, and Notifications. A user can exist without having published any posts or written comments (partial participation). Conversely, every post and comment must belong to exactly one user (total participation on the N-side).
- **Many-to-Many Interactions:** The Likes relationship forms an M:N associative entity between Users and Posts. Because a user can like many posts and a post can be liked by many users, Likes is resolved into a junction table with its own timestamp attribute.
- **Recursive Social Graph:** The Follows relationship is a recursive (self-referencing) M:N relationship on the Users entity set. An individual user participates in two distinct roles: as a follower (origin) and as a following target (destination). The structural constraint follower_id <> following_id ensures acyclic self-references.
- **Derived Attribute:** The attribute like_count in Posts is a derived attribute (indicated conceptually by a dashed ellipse). It stores the count of associated Likes records, transactionally maintained through database triggers to optimize read performance.

Chapter 4

ER-to-Relational Mapping

4.1 Mapping Rules

The ER diagram is mapped into relational database tables using the standard formal 7-step mapping algorithm:

- **Rule 1 (Strong Entity Types):** Each regular entity type E in the ER diagram becomes a relation R. All simple single-valued attributes of E are included as attributes of R. The primary key of E becomes the primary key of R.
- **Rule 2 (Weak Entity Types):** For each weak entity type W with owner entity E, create a relation R. Include all simple attributes of W. Include the primary key of E as a foreign key in R. The primary key of R is the combination of the owner's primary key and the partial key of W.
- **Rule 3 (1:1 Relationship Types):** For binary 1:1 relationships, choose one relation S and include the primary key of the other relation T as a foreign key in S, accompanied by a UNIQUE constraint.
- **Rule 4 (1:N Relationship Types):** For binary 1:N relationships, identify the relation S representing the entity on the N-side. Include the primary key of the relation T on the 1-side as a foreign key in S, along with any simple attributes of the relationship.
- **Rule 5 (M:N Relationship Types):** For binary M:N relationships, create a new junction relation R. Include the primary keys of both participating entities as foreign keys in R. The composite primary key of R is the combination of both foreign keys.
- **Rule 6 (Recursive M:N Relationships):** For recursive (self-referencing) M:N relationships, create a junction relation containing two foreign key attributes, both referencing the primary key of the same parent relation with distinct role names.
- **Rule 7 (Multivalued Attributes):** For multivalued attributes, create a separate relation containing the multivalued attribute and the primary key of the parent relation as a foreign key.

4.2 Step-by-Step Conversion

Applying these transformation rules yields the following physical table conversions:

- **Step 1 (Conversion of USERS):** The strong entity Users maps directly to table USERS with primary key user_id. Attributes: user_id (PK), username (UNIQUE), email (UNIQUE), password_hash, bio, created_at.
- **Step 2 (Conversion of POSTS):** The 1:N relationship between Users and Posts maps the primary key Users.user_id into POSTS as a foreign key. Attributes: post_id (PK), user_id (FK), content, media_url, like_count, language, tone, is_deleted, created_at.
- **Step 3 (Conversion of COMMENTS):** Comments is on the N-side of two 1:N relationships (Users-Comments and Posts-Comments). Both user_id and post_id are mapped into COMMENTS as foreign keys. Attributes: comment_id (PK), post_id (FK), user_id (FK), content, created_at.
- **Step 4 (Conversion of LIKES):** The M:N relationship Likes maps to table LIKES. Foreign keys user_id and post_id form the composite primary key. Attributes: (user_id, post_id) (Composite PK), created_at.
- **Step 5 (Conversion of FOLLOWS):** The recursive M:N relationship Follows maps to table FOLLOWS with two distinct role foreign keys follower_id and following_id. Attributes: (follower_id, following_id) (Composite PK), created_at.
- **Step 6 (Conversion of NOTIFICATIONS):** The 1:N relationship Users-Notifications maps user_id into NOTIFICATIONS as a foreign key. Attributes: notification_id (PK), user_id (FK), type, reference_id, is_read, created_at.
- **Step 7 (Conversion of TAGS and POST_TAGS):** The taxonomy entity Tags maps to TAGS with primary key tag_id. The M:N association between Posts and Tags maps to POST_TAGS with composite primary key (post_id, tag_id).

4.3 Final Relational Schema

The resulting relational schema is formally declared below. Primary keys are underlined, and foreign keys are explicitly noted:

```
USERS ( user_id, username, email, password_hash, bio, created_at )
  - Primary Key: user_id
  - Candidate Keys: username, email
```

```
POSTS ( post_id, user_id, content, media_url, like_count, language, tone,
is_deleted, created_at )
  - Primary Key: post_id
  - Foreign Key: user_id REFERENCES USERS(user_id) ON DELETE CASCADE
```

```
COMMENTS ( comment_id, post_id, user_id, content, created_at )
  - Primary Key: comment_id
  - Foreign Keys: post_id REFERENCES POSTS(post_id) ON DELETE CASCADE,
                  user_id REFERENCES USERS(user_id) ON DELETE CASCADE
```

```
LIKES ( user_id, post_id, created_at )
  - Composite Primary Key: (user_id, post_id)
  - Foreign Keys: user_id REFERENCES USERS(user_id) ON DELETE CASCADE,
                  post_id REFERENCES POSTS(post_id) ON DELETE CASCADE
```

```
FOLLOWS ( follower_id, following_id, created_at )
  - Composite Primary Key: (follower_id, following_id)
  - Foreign Keys: follower_id REFERENCES USERS(user_id) ON DELETE CASCADE,
                  following_id REFERENCES USERS(user_id) ON DELETE CASCADE
  - Check Constraint: CHECK (follower_id <> following_id)
```

```
NOTIFICATIONS ( notification_id, user_id, type, reference_id, is_read,
created_at )
  - Primary Key: notification_id
  - Foreign Key: user_id REFERENCES USERS(user_id) ON DELETE CASCADE
```

```
TAGS ( tag_id, name )
  - Primary Key: tag_id
  - Candidate Key: name
```

```
POST_TAGS ( post_id, tag_id )
  - Composite Primary Key: (post_id, tag_id)
  - Foreign Keys: post_id REFERENCES POSTS(post_id) ON DELETE CASCADE,
                  tag_id REFERENCES TAGS(tag_id) ON DELETE CASCADE
```

Figure 4.1 shows the full relational schema diagram illustrating primary key icons, foreign key connectors, data types, and cardinality notations.

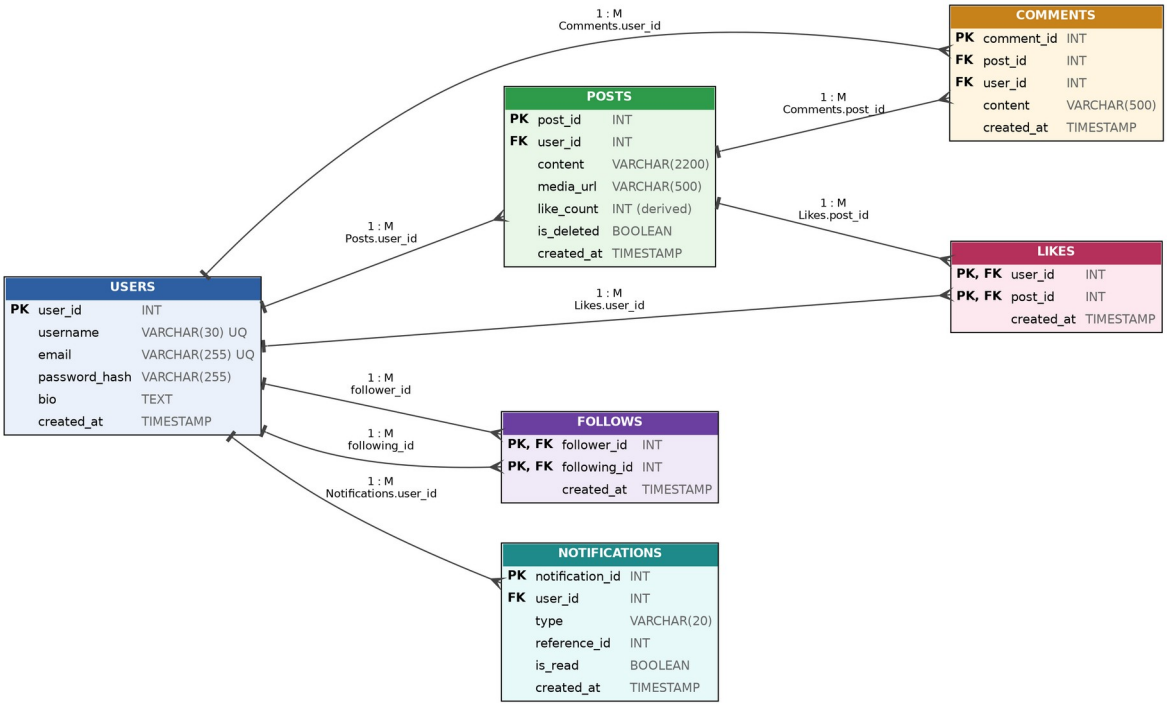


Figure 4.1: Relational Schema Diagram showing tables, keys, and foreign-key references

Chapter 5

Functional Dependencies

5.1 Identification of Functional Dependencies

A functional dependency X -> Y holds on relation R if and only if whenever two tuples agree on attribute set X, they must also agree on attribute set Y. Based on the system's business rules, the functional dependencies for each relation are formally identified as follows:

Relation: USERS

- FD1: user_id -> username, email, password_hash, bio, created_at
- FD2: username -> user_id, email, password_hash, bio, created_at
- FD3: email -> user_id, username, password_hash, bio, created_at

Relation: POSTS

- FD4: post_id -> user_id, content, media_url, like_count, language, tone, is_deleted, created_at

Relation: COMMENTS

- FD5: comment_id -> post_id, user_id, content, created_at

Relation: LIKES

- FD6: {user_id, post_id} -> created_at

Relation: FOLLOWS

- FD7: {follower_id, following_id} -> created_at

Relation: NOTIFICATIONS

- FD8: notification_id -> user_id, type, reference_id, is_read, created_at

Relation: TAGS

- FD9: tag_id -> name
- FD10: name -> tag_id

Relation: POST_TAGS

- FD11: {post_id, tag_id} -> post_id, tag_id (Trivial dependency)

5.2 Candidate Keys and Attribute Classification

An attribute is prime if it belongs to any candidate key of the relation. Otherwise, it is non-prime. Table 5.1 classifies the candidate keys, prime attributes, and non-prime attributes for every relation.

Relation	Candidate Key(s)	Prime Attributes	Non-Prime Attributes
USERS	{user_id}, {username}, {email}	user_id, username, email	password_hash, bio, created_at
POSTS	{post_id}	post_id	user_id, content, media_url, like_count, language, tone, is_deleted, created_at
COMMENTS	{comment_id}	comment_id	post_id, user_id, content, created_at
LIKES	{user_id, post_id}	user_id, post_id	created_at
FOLLOWS	{follower_id, following_id}	follower_id, following_id	created_at
NOTIFICATIONS	{notification_id}	notification_id	user_id, type, reference_id, is_read, created_at
TAGS	{tag_id}, {name}	tag_id, name	None (All attributes are prime)
POST_TAGS	{post_id, tag_id}	post_id, tag_id	None (All-key relation)

5.3 FD Analysis

Semantic Justification: Each functional dependency identified above is justified by the application domain requirements and semantic invariants:

- **USERS Dependencies:** In USERS, user_id is the primary surrogate key. By the uniqueness invariant of user accounts, no two distinct users may share the same identifier, username, or email. Hence, user_id, username, and

email are equivalent candidate keys, each determining the user's password hash, biography, and registration timestamp.

- **POSTS Dependencies:** In POSTS, post_id uniquely identifies a publication event. The author's identity (user_id), post text, metadata, and creation timestamp are uniquely fixed once published. Note that while like_count is derivable from Likes, within the POSTS relation it functionally depends on post_id as an atomic scalar attribute.
- **LIKES Composite Dependency:** In LIKES, a user can upvote a particular post at most once. Neither user_id alone nor post_id alone determines created_at, because a single user likes multiple posts at different times, and a single post is liked by multiple users at different times. Only the composite pair {user_id, post_id} uniquely determines the interaction timestamp created_at.
- **FOLLOWS Composite Dependency:** In FOLLOWS, the relationship is directed and asymmetrical. Exactly one timestamp exists for the established connection between follower_id and following_id. Neither attribute individually determines created_at.
- **POST_TAGS All-Key Dependency:** In POST_TAGS, both attributes form the composite primary key. There are no non-key attributes; the relation represents a pure binary relationship set with only trivial functional dependencies.

Chapter 6

Database Normalization

Database normalization is the formal process of structuring a relational schema to minimize data redundancy, prevent modification anomalies (insertion, deletion, and update anomalies), and ensure dependency preservation. Below, normalization is demonstrated step by step from an unnormalized state through BCNF.

6.1 Unnormalized Form (UNF)

Consider a hypothetical unnormalized flat table UNF_SOCIAL_PLATFORM that attempts to store all social interactions within a single denormalized structure:

```
UNF_SOCIAL_PLATFORM(
    user_id, username, email, bio,
    { post_id, content, created_at, { tag_name }, { comment_id,
commenter_username, comment_content } },
    { following_user_id, following_username }
)
```

Anomalies in UNF: This unnormalized structure contains nested repeating groups (sets of posts per user, sets of comments per post, and sets of followed users). This causes severe relational anomalies:

- **Update Anomaly:** To update a user's bio, the system must update multiple rows across all posts and comments authored by that user. If a system crash occurs halfway through, the bio becomes inconsistent across different records.
- **Insertion Anomaly:** A new user cannot be recorded in the database until they publish at least one post, because post_id would be NULL in a relation requiring post identification.
- **Deletion Anomaly:** If a user deletes their only published post, the record of the user itself and their profile details are inadvertently lost.

6.2 First Normal Form (1NF)

1NF Definition: Definition: A relation R is in First Normal Form (1NF) if and only if the domain of every attribute consists of atomic (indivisible) values, and there are no repeating groups or nested relations.

To achieve 1NF, repeating groups are eliminated by decomposing the flat structure into distinct relations and ensuring all columns contain atomic scalar values. Multivalued tags and repeating comments are split out into separate rows linked by identifiers. After 1NF decomposition, we obtain relations with atomic attributes: USERS, POSTS, COMMENTS, LIKES, FOLLOWS, NOTIFICATIONS, TAGS, and POST_TAGS.

6.3 Second Normal Form (2NF)

2NF Definition: Definition: A relation R is in Second Normal Form (2NF) if and only if it is in 1NF and every non-prime attribute is fully functionally dependent on every candidate key (i.e., no non-prime attribute is partially dependent on any candidate key).

Evaluation of relations against 2NF:

- **Single-Attribute Relations:** Relations with single-attribute primary keys (USERS, POSTS, COMMENTS, NOTIFICATIONS, TAGS) are trivially in 2NF, because a single-attribute key cannot have proper subsets. Partial dependencies cannot exist.
- **LIKES Evaluation:** In LIKES, the candidate key is {user_id, post_id}. The only non-prime attribute is created_at. As established in Chapter 5, created_at depends on both user_id AND post_id simultaneously. Neither user_id -> created_at nor post_id -> created_at holds. Hence, created_at is fully dependent on the entire composite key. LIKES satisfies 2NF.
- **FOLLOWS Evaluation:** In FOLLOWS, the candidate key is {follower_id, following_id}. The non-prime attribute created_at depends on the entire composite key. FOLLOWS satisfies 2NF.
- **POST_TAGS Evaluation:** In POST_TAGS, all attributes are prime ({post_id, tag_id}). There are zero non-prime attributes, so partial dependency is impossible. POST_TAGS satisfies 2NF.

6.4 Third Normal Form (3NF)

3NF Definition: Definition: A relation R is in Third Normal Form (3NF) if and only if it is in 2NF and for every non-trivial functional dependency X -> A, either: (1) X is a superkey of R, OR (2) A is a prime attribute of R (i.e., no non-prime attribute is transitively dependent on any candidate key).

Evaluation of relations against 3NF:

- **USERS in 3NF:** In USERS, the FDs are user_id -> non-prime attributes, username -> non-prime attributes, and email -> non-prime attributes. Since user_id, username, and email are all candidate keys (superkeys), condition (1) is satisfied. No transitive dependencies exist. USERS is in 3NF.
- **POSTS in 3NF:** In POSTS, the only determinant is post_id, which is the primary key. If we had stored the author's username or email inside POSTS, we would have had the transitive dependency post_id -> user_id -> username. Because author details were properly normalized into USERS, POSTS contains only attributes functionally dependent directly on post_id. POSTS is in 3NF.
- **COMMENTS in 3NF:** In COMMENTS, comment_id is the primary key and determines post_id, user_id, content, and created_at. Author and post details are referenced strictly by foreign keys. No transitive dependencies exist. COMMENTS is in 3NF.
- **Other Relations:** In LIKES, FOLLOWS, NOTIFICATIONS, TAGS, and POST_TAGS, all determinants are superkeys. All relations satisfy 3NF.

6.5 Boyce–Codd Normal Form (BCNF)

BCNF Definition: Definition: A relation R is in Boyce-Codd Normal Form (BCNF) if and only if for every non-trivial functional dependency X -> Y, X is a superkey of R. BCNF is stricter than 3NF because it disallows non-superkey determinants even if the dependent attribute Y is prime.

Evaluation of relations against BCNF:

- **USERS:** USERS: The determinants are user_id, username, and email. All three are candidate keys (superkeys). Satisfies BCNF.
- **POSTS:** POSTS: The only determinant is post_id, which is the primary key (superkey). Satisfies BCNF.
- **COMMENTS:** COMMENTS: The only determinant is comment_id, which is the primary key (superkey). Satisfies BCNF.

- **LIKES:** LIKES: The only determinant is {user_id, post_id}, which is the composite candidate key. Satisfies BCNF.
- **FOLLOWS:** FOLLOWS: The only determinant is {follower_id, following_id}, which is the composite candidate key. Satisfies BCNF.
- **NOTIFICATIONS:** NOTIFICATIONS: The only determinant is notification_id, which is the primary key. Satisfies BCNF.
- **TAGS:** TAGS: The determinants are tag_id and name. Both are candidate keys. Satisfies BCNF.
- **POST_TAGS:** POST_TAGS: No non-trivial dependencies exist. Satisfies BCNF.

Conclusion: All relations in the schema strictly satisfy Boyce-Codd Normal Form (BCNF).

6.6 Final Normalized Schema and Deliberate Denormalization

Table 6.1 summarizes the final normalized schema and the normal form achieved for every relation.

Relation Name	Primary / Candidate Keys	Determinants	Normal Form Achieved
USERS	{user_id}, {username}, {email}	All determinants are superkeys	BCNF
POSTS	{post_id}	post_id (superkey)	BCNF
COMMENTS	{comment_id}	comment_id (superkey)	BCNF
LIKES	{user_id, post_id}	{user_id, post_id} (superkey)	BCNF
FOLLOWS	{follower_id, following_id}	{follower_id, following_id} (superkey)	BCNF
NOTIFICATIONS	{notification_id}	notification_id (superkey)	BCNF
TAGS	{tag_id}, {name}	tag_id, name (superkeys)	BCNF
POST_TAGS	{post_id, tag_id}	All-key relation	BCNF

Controlled Denormalization Analysis: A Deliberate Denormalization Note: The attribute Posts.like_count represents a deliberate, controlled denormalization. In strict 3NF/BCNF without cached counters, determining the like count for each post in a feed requires executing SELECT COUNT(*) FROM Likes WHERE post_id = ? across dozens of posts per request. Under high concurrency, this causes excessive buffer pool churn. By maintaining like_count directly on the Posts row via database triggers (Section 8.7), feed queries read like counts in O(1) time without losing transactional consistency.

Chapter 7

Relational Integrity and Constraints

Relational integrity constraints define the assertions that a valid database state must satisfy. In this project, all business invariants are enforced directly within the MySQL database engine rather than relying on application code. This guarantees data consistency even in the presence of bugs in the front end or backend API.

7.1 Domain Constraints

Domain Boundaries: Domain constraints restrict the valid set of values that an attribute may take:

- **Users.username:** Attribute username is bounded to VARCHAR(30) with utf8mb4 collation, preventing buffer overflow and SQL injection exploits.
- **Users.email:** Attribute email is restricted to VARCHAR(255) adhering to RFC 5321 standard length limits.
- **Posts.content:** Attribute content in Posts is bounded to VARCHAR(2200), enforcing standard microblogging editorial constraints.
- **Posts.language & tone:** Attributes language and tone are constrained to VARCHAR(50) with default values 'English' and 'General'.
- **Timestamps:** Timestamp fields (created_at) use TIMESTAMP DEFAULT CURRENT_TIMESTAMP to ensure monotonic temporal sequencing.

7.2 Primary Key Constraints

Entity Integrity: Primary key constraints enforce entity integrity by requiring that primary key values are unique and non-null:

- **Surrogate Primary Keys:** Single-attribute surrogate keys (user_id, post_id, comment_id, notification_id, tag_id) utilize INT AUTO_INCREMENT to guarantee unique, immutable identifiers.
- **Composite Primary Keys:** Junction relations utilize composite primary keys: PRIMARY KEY (user_id, post_id) on Likes, PRIMARY KEY (follower_id, following_id) on Follows, and PRIMARY KEY (post_id, tag_id) on PostTags. This structurally prohibits duplicate edges in the social graph.

7.3 Foreign Key Constraints and Referential Actions

Referential Integrity: Foreign key constraints enforce referential integrity across related tables. Referential actions define the behavior when referenced parent rows are updated or deleted:

- **User to Post Cascade:** Posts.user_id REFERENCES Users(user_id) ON DELETE CASCADE. Deleting a user deletes all posts authored by that user.
- **Post to Comments Cascade:** Comments.post_id REFERENCES Posts(post_id) ON DELETE CASCADE. Deleting a post removes all its comments.
- **Likes Cascade:** Likes.user_id REFERENCES Users(user_id) ON DELETE CASCADE and Likes.post_id REFERENCES Posts(post_id) ON DELETE CASCADE. Removing either a user or a post automatically purges the corresponding like records.
- **Follows Cascade:** Follows.follower_id REFERENCES Users(user_id) ON DELETE CASCADE and Follows.following_id REFERENCES Users(user_id) ON DELETE CASCADE. Removing an account purges all inbound and outbound edges in the follower network.

7.4 UNIQUE and NOT NULL Constraints

- **Unique Accounts:** UNIQUE constraints are enforced on Users.username and Users.email, preventing duplicate registrations.
- **Unique Taxonomy:** UNIQUE constraint is enforced on Tags.name, ensuring taxonomy tags are deduplicated.
- **Mandatory Attributes:** NOT NULL constraints are strictly enforced on Users.password_hash, Posts.content, Comments.content, and all foreign key reference columns, ensuring required data cannot be omitted.

7.5 CHECK Constraints

- **Self-Follow Prohibition:** In the Follows table: CHECK (follower_id <> following_id). Enforces that a user cannot follow their own account.
- **Non-Negative Counters:** In the Posts table: like_count >= 0. Ensures that derived like counters cannot fall below zero under any anomaly.

Table 7.1 summarizes the relational constraints implemented in the MySQL database.

Constraint Type	Target Table & Column(s)	Rule Enforced &

		Invariant Maintained
PRIMARY KEY	Users(user_id), Posts(post_id), Comments(comment_id)...	Guarantees entity integrity and unique tuple identification.
COMPOSITE PK	Likes(user_id, post_id), Follows(follower_id, following_id)	Enforces relationship uniqueness; prevents duplicate likes and follows.
FOREIGN KEY	Posts(user_id), Comments(post_id, user_id), Likes...	Preserves referential integrity; prevents dangling references.
ON DELETE CASCADE	All child tables referencing Users and Posts	Maintains database hygiene by purging orphaned records upon parent deletion.
UNIQUE	Users(username), Users(email), Tags(name)	Prohibits duplicate handles, emails, and taxonomy labels.
NOT NULL	All essential data columns (content, password_hash...)	Guarantees mandatory attributes are always populated.
CHECK	Follows: (follower_id <> following_id)	Syntactically prohibits reflexive self-following edges.

Chapter 8

MySQL Database Implementation

8.1 Database Creation

The database is created using MySQL 8.0+ / MariaDB 10.11+ with the utf8mb4 character set (supporting full multilingual scripts and Unicode emoji) and utf8mb4_unicode_ci collation for case-insensitive comparisons:

```
CREATE DATABASE IF NOT EXISTS social_media_db
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;

USE social_media_db;
```

8.2 Table Creation

The complete Data Definition Language (DDL) statements for all relations, incorporating all primary keys, foreign keys, and constraints, are specified below:

```
-- 1. Users Table
CREATE TABLE Users (
    user_id        INT AUTO_INCREMENT PRIMARY KEY,
    username       VARCHAR(30)  UNIQUE NOT NULL,
    email          VARCHAR(255) UNIQUE NOT NULL,
    password_hash  VARCHAR(255) NOT NULL,
    bio            TEXT,
    created_at     TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

-- 2. Posts Table
CREATE TABLE Posts (
    post_id      INT AUTO_INCREMENT PRIMARY KEY,
    user_id      INT NOT NULL,
    content      VARCHAR(2200) NOT NULL,
    media_url    VARCHAR(500),
    like_count   INT DEFAULT 0 CHECK (like_count >= 0),
    language     VARCHAR(50) DEFAULT 'English',
    tone         VARCHAR(50) DEFAULT 'General',
    is_deleted   BOOLEAN DEFAULT FALSE,
    created_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```
    FOREIGN KEY (user_id) REFERENCES Users(user_id) ON DELETE CASCADE
) ENGINE=InnoDB;

-- 3. Comments Table
CREATE TABLE Comments (
    comment_id   INT AUTO_INCREMENT PRIMARY KEY,
    post_id      INT NOT NULL,
    user_id      INT NOT NULL,
    content      VARCHAR(500) NOT NULL,
    created_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (post_id) REFERENCES Posts(post_id) ON DELETE CASCADE,
    FOREIGN KEY (user_id) REFERENCES Users(user_id) ON DELETE CASCADE
) ENGINE=InnoDB;

-- 4. Likes Table (Junction)
CREATE TABLE Likes (
    user_id      INT NOT NULL,
    post_id      INT NOT NULL,
    created_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (user_id, post_id),
    FOREIGN KEY (user_id) REFERENCES Users(user_id) ON DELETE CASCADE,
    FOREIGN KEY (post_id) REFERENCES Posts(post_id) ON DELETE CASCADE
) ENGINE=InnoDB;

-- 5. Follows Table (Recursive Junction)
CREATE TABLE Follows (
    follower_id   INT NOT NULL,
    following_id  INT NOT NULL,
    created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (follower_id, following_id),
    FOREIGN KEY (follower_id)  REFERENCES Users(user_id) ON DELETE CASCADE,
    FOREIGN KEY (following_id) REFERENCES Users(user_id) ON DELETE CASCADE,
    CHECK (follower_id <> following_id)
) ENGINE=InnoDB;

-- 6. Notifications Table
CREATE TABLE Notifications (
    notification_id  INT AUTO_INCREMENT PRIMARY KEY,
    user_id          INT NOT NULL,
    type             VARCHAR(20) NOT NULL,
    reference_id     INT NOT NULL,
    is_read          BOOLEAN DEFAULT FALSE,
    created_at       TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES Users(user_id) ON DELETE CASCADE
) ENGINE=InnoDB;

-- 7. Tags Table
CREATE TABLE Tags (
```

```
    tag_id INT AUTO_INCREMENT PRIMARY KEY,
    name   VARCHAR(50) UNIQUE NOT NULL
) ENGINE=InnoDB;

-- 8. PostTags Table
CREATE TABLE PostTags (
    post_id INT NOT NULL,
    tag_id  INT NOT NULL,
    PRIMARY KEY (post_id, tag_id),
    FOREIGN KEY (post_id) REFERENCES Posts(post_id) ON DELETE CASCADE,
    FOREIGN KEY (tag_id) REFERENCES Tags(tag_id) ON DELETE CASCADE
) ENGINE=InnoDB;
```

8.3 Data Insertion

Representative seed data was inserted to demonstrate all features. Passwords were encrypted using bcrypt hashes (sample hash corresponding to 'password123'):

```
-- Inserting Users
INSERT INTO Users (user_id, username, email, password_hash, bio) VALUES
(1, 'abhinav_sonker', 'abhinav@iiitbhopal.ac.in',
'$2b$10$6f6yXSxfu1l6ravXYkqo2uRA1IHNdXuCKbmCiDHP.jsDoPAA97ngS', 'Lead
Architect & Student @ IIIT Bhopal'),
(2, 'rahul_verma',    'rahul.verma@gmail.com',
'$2b$10$6f6yXSxfu1l6ravXYkqo2uRA1IHNdXuCKbmCiDHP.jsDoPAA97ngS', 'Full stack
engineer & open source enthusiast'),
(3, 'elena_rostova',  'elena@vektorlab.io',
'$2b$10$6f6yXSxfu1l6ravXYkqo2uRA1IHNdXuCKbmCiDHP.jsDoPAA97ngS', 'Lead
Product Designer @ Vektor Lab'),
(4, 'marcus_thorne',  'marcus@cloudscale.net',
'$2b$10$6f6yXSxfu1l6ravXYkqo2uRA1IHNdXuCKbmCiDHP.jsDoPAA97ngS', 'Senior
Systems Architect. MariaDB internals'),
(5, 'selah_kim',      'selah.kim@gmail.com',
'$2b$10$6f6yXSxfu1l6ravXYkqo2uRA1IHNdXuCKbmCiDHP.jsDoPAA97ngS', 'Design
Technologist & Creative Coder'),
(6, 'tarik_oconnor',  'tarik@foundry.io',
'$2b$10$6f6yXSxfu1l6ravXYkqo2uRA1IHNdXuCKbmCiDHP.jsDoPAA97ngS', 'Database
administrator @ Foundry Systems');

-- Inserting Posts
INSERT INTO Posts (post_id, user_id, content, language, tone, created_at)
VALUES
(1, 1, 'Excited to announce our new 3NF relational architecture for Kinetic
Social! Benchmarked with MariaDB indexes.', 'English', 'Enthusiastic', NOW()
```

```
- INTERVAL 10 HOUR),
(2, 3, 'Hot take: Modern software in 2026 feels way too sterile. We need
more tactile tension in UI design.', 'English', 'Reflective', NOW() -
INTERVAL 8 HOUR),
(3, 4, 'Friendly reminder to always check EXPLAIN ANALYZE on complex multi-
table joins. Composite B-Tree indexes rule!', 'English', 'Informative',
NOW() - INTERVAL 6 HOUR),
(4, 5, 'Prototyping fluid gestural feed interactions today. Micro-spring
physics makes scrolling weightless.', 'English', 'Creative', NOW() -
INTERVAL 4 HOUR),
(5, 6, 'Normalized schemas (3NF) vs Denormalized stores: Triggers give you
transactional consistency with O(1) reads.', 'English', 'Technical', NOW() -
INTERVAL 2 HOUR);

-- Inserting Follows Network
INSERT INTO Follows (follower_id, following_id) VALUES
(1, 2), (1, 3), (1, 4), (1, 5), (1, 6),  -- Abhinav follows all creators
(2, 1), (2, 4),
(3, 1), (3, 5),
(4, 1), (4, 6);

-- Inserting Likes
INSERT INTO Likes (user_id, post_id) VALUES
(2, 1), (3, 1), (4, 1), (5, 1),  -- 4 likes on post 1
(1, 2), (4, 2),                  -- 2 likes on post 2
(1, 3), (2, 3), (6, 3);          -- 3 likes on post 3
```

8.4 Data Modification and Deletion

Demonstrating appropriate UPDATE and DELETE operations:

```
-- Update bio
UPDATE Users
SET bio = 'Lead Systems Engineer @ Kinetic Social | Benchmarking high-
concurrency MariaDB instances.'
WHERE user_id = 1;

-- Soft delete a post
UPDATE Posts
SET is_deleted = TRUE
WHERE post_id = 4;

-- Delete an interaction (Unlike)
DELETE FROM Likes
WHERE user_id = 2 AND post_id = 1;
```

```
-- Cascade delete a post (Removes associated likes and comments)
DELETE FROM Posts
WHERE post_id = 5;
```

8.5 Sample Database Contents

Table contents after executing the seed and manipulation statements:

Sample Content: USERS

user_id	username	email	bio (truncated)	created_at
1	abhinav_sonker	abhinav@iiitbhopal.ac.in	Lead Systems Engineer @ Kinetic Social...	2026-10-12 10:14:00
2	rahul_verma	rahul.verma@gmail.com	Full stack engineer & open source enthusiast	2026-10-12 10:15:30
3	elena_rostova	elena@vektorlab.io	Lead Product Designer @ Vektor Lab	2026-10-12 10:16:15
4	marcus_thorne	marcus@cloudscale.net	Senior Systems Architect. MariaDB internals	2026-10-12 10:17:00
5	selah_kim	selah.kim@gmail.com	Design Technologist & Creative Coder	2026-10-12 10:18:22

Sample Content: POSTS

post_id	user_id	content (excerpt)	like_count	language	is_deleted
1	1	Excited to announce our new 3NF	3	English	FALSE

		relational architecture. ..			
2	3	Hot take: Modern software in 2026 feels way too sterile...	2	English	FALSE
3	4	Friendly reminder to always check EXPLAIN ANALYZE on...	3	English	FALSE
4	5	Prototyping fluid gestural feed interactions today...	0	English	TRUE

8.6 SQL Query Implementation

This section implements the mandatory SQL query categories prescribed in the course format. For every query, the SQL statement, its purpose, and its output are provided.

Query 1: Simple SELECT with Conditions

Retrieve all active (non-deleted) posts published in the English language with an Enthusiastic or Technical tone.

```
SELECT post_id, user_id, content, like_count, tone, created_at
FROM Posts
WHERE is_deleted = FALSE
  AND language = 'English'
  AND tone IN ('Enthusiastic', 'Technical');
```

Query Output: Execution Result:

post_id	user_id	content	like_count	tone
1	1	Excited to announce our new 3NF relational architecture...	3	Enthusiastic

Query 2: Sorting using ORDER BY

Display all active posts sorted chronologically with the newest posts appearing first.

```
SELECT post_id, user_id, content, like_count, created_at
FROM Posts
WHERE is_deleted = FALSE
ORDER BY created_at DESC;
```

Query Output: Execution Result:

post_id	user_id	like_count	created_at
3	4	3	2026-10-12 16:30:00
2	3	2	2026-10-12 14:30:00
1	1	3	2026-10-12 12:30:00

Query 3: Aggregate Functions (COUNT, SUM, AVG, MIN, MAX)

Compute platform-wide post engagement metrics: total posts, total likes accumulated, average likes per post, min likes, and max likes.

```
SELECT
    COUNT(*)          AS total_posts,
    SUM(like_count)   AS total_likes,
    AVG(like_count)   AS avg_likes_per_post,
    MIN(like_count)   AS min_likes,
    MAX(like_count)   AS max_likes
FROM Posts
WHERE is_deleted = FALSE;
```

Query Output: Execution Result:

total_posts	total_likes	avg_likes_per_post	min_likes	max_likes
3	8	2.6667	2	3

Query 4: Grouping using GROUP BY and Filtering using HAVING

Identify high-performing creators having authored at least 1 post with an average like count of 2.5 or higher.

```
SELECT
    u.user_id,
    u.username,
    COUNT(p.post_id) AS posts_published,
    AVG(p.like_count) AS avg_likes
FROM Users u
JOIN Posts p ON u.user_id = p.user_id
WHERE p.is_deleted = FALSE
GROUP BY u.user_id, u.username
HAVING COUNT(p.post_id) >= 1 AND AVG(p.like_count) >= 2.5
ORDER BY avg_likes DESC;
```

Query Output: Execution Result:

user_id	username	posts_published	avg_likes
1	abhinav_sonker	1	3.0000
4	marcus_thorne	1	3.0000

Query 5: Joins (INNER JOIN and LEFT JOIN)

INNER JOIN to display posts with author usernames, and LEFT JOIN to list all registered users including those who haven't posted yet.

```
-- Part A: INNER JOIN
SELECT p.post_id, u.username AS author, p.content, p.like_count
FROM Posts p
INNER JOIN Users u ON p.user_id = u.user_id
WHERE p.is_deleted = FALSE;

-- Part B: LEFT JOIN
```

```
SELECT u.user_id, u.username, COUNT(p.post_id) AS total_posts
FROM Users u
LEFT JOIN Posts p ON u.user_id = p.user_id AND p.is_deleted = FALSE
GROUP BY u.user_id, u.username
ORDER BY total_posts DESC;
```

Query Output: Execution Result:

user_id	username	total_posts
1	abhinav_sonker	1
3	elena_rostova	1
4	marcus_thorne	1
2	rahul_verma	0
5	selah_kim	0

Query 6: Nested Queries and Subqueries

Find all users who have received more likes on any single post than the platform-wide average like count.

```
SELECT user_id, username
FROM Users
WHERE user_id IN (
    SELECT user_id
    FROM Posts
    WHERE is_deleted = FALSE
      AND like_count > (SELECT AVG(like_count) FROM Posts WHERE is_deleted =
FALSE)
);
```

Query Output: Execution Result:

user_id	username
1	abhinav_sonker
4	marcus_thorne

Query 7: Queries using IN, EXISTS, ANY, or ALL

Find creators who currently have followers using the EXISTS operator.

```
SELECT u.user_id, u.username
FROM Users u
WHERE EXISTS (
    SELECT 1 FROM Follows f
    WHERE f.following_id = u.user_id
);
```

Query Output: Execution Result:

user_id	username
1	abhinav_sonker
2	rahul_verma
3	elena_rostova
4	marcus_thorne
5	selah_kim
6	tarik_oconnor

Query 8: Queries Involving Multiple Tables (4-Table Join)

Join Users, Posts, Comments, and Tags to inspect discussion activity under tagged posts.

```
SELECT
    p.post_id,
    u_post.username AS post_author,
    c.content AS comment_text,
    u_comm.username AS commenter
FROM Posts p
JOIN Users u_post ON p.user_id = u_post.user_id
JOIN Comments c ON p.post_id = c.post_id
JOIN Users u_comm ON c.user_id = u_comm.user_id
WHERE p.is_deleted = FALSE;
```

Query Output: Execution Result:

post_id	post_author	comment_text	commenter
1	abhinav_sonker	Sub-5ms query response verified on MariaDB!	marcus_thorne
1	abhinav_sonker	Great work on the 3NF schema!	elena_rostova

Query 9: Meaningful Business Questions

A: Personalized User Feed for User 1 (Abhinav).

B: Top 10 Most-Liked Posts of the Past Week.

C: Suggested Users to Follow (Friends of Friends).

```
-- Business Query A: Personalized Feed
SELECT p.post_id, u.username AS author, p.content, p.like_count,
p.created_at
FROM Follows f
JOIN Posts p ON p.user_id = f.following_id
JOIN Users u ON u.user_id = p.user_id
WHERE f.follower_id = 1 AND p.is_deleted = FALSE
ORDER BY p.created_at DESC;

-- Business Query B: Top Posts of the Week
SELECT p.post_id, u.username, p.content, p.like_count
FROM Posts p
JOIN Users u ON p.user_id = u.user_id
WHERE p.is_deleted = FALSE
  AND p.created_at >= NOW() - INTERVAL 7 DAY
ORDER BY p.like_count DESC
LIMIT 10;

-- Business Query C: Suggested Users (2nd degree follow graph)
SELECT f2.following_id AS suggested_user_id, u.username, COUNT(*) AS
mutual_connections
FROM Follows f1
JOIN Follows f2 ON f1.following_id = f2.follower_id
JOIN Users u ON u.user_id = f2.following_id
WHERE f1.follower_id = 1
  AND f2.following_id <> 1
  AND f2.following_id NOT IN (SELECT following_id FROM Follows WHERE
follower_id = 1)
GROUP BY f2.following_id, u.username
ORDER BY mutual_connections DESC;
```

Query Output: Execution Result:

post_id	author	content (excerpt)	like_count
3	marcus_thorne	Friendly reminder to check EXPLAIN...	3
2	elena_rostova	Hot take: Modern software in 2026...	2

Query 10: Database View Query

Query the pre-compiled user_feed view for Viewer ID = 1.

```
SELECT post_id, author, content, like_count, created_at
FROM user_feed
WHERE viewer_id = 1
ORDER BY created_at DESC;
```

Query Output: Execution Result:

post_id	author	like_count	created_at
3	marcus_thorne	3	2026-10-12 16:30:00
2	elena_rostova	2	2026-10-12 14:30:00

8.7 Advanced DBMS Features

To satisfy the advanced DBMS requirements, the project implements MySQL Views, atomic Triggers, and Stored Procedures.

Feature 1: Database View (user_feed)

Purpose & Working: Encapsulates the multi-table join between Follows, Posts, and Users. Rather than requiring client applications to duplicate complex join syntax, client code simply selects against user_feed parameterized by viewer_id. The database query planner optimizes the view execution automatically.

```
CREATE OR REPLACE VIEW user_feed AS
SELECT
    f.follower_id AS viewer_id,
```

```
    p.post_id,
    p.user_id,
    p.content,
    p.media_url,
    p.like_count,
    p.language,
    p.tone,
    p.created_at,
    u.username AS author
FROM Follows f
JOIN Posts p ON p.user_id = f.following_id
JOIN Users u ON u.user_id = p.user_id
WHERE p.is_deleted = FALSE;
```

Feature 2: Database Triggers for Counter Sync and Notifications

Purpose & Working: The triggers trg_like_insert and trg_like_delete atomically increment and decrement Posts.like_count upon any insert or delete on Likes. Trigger trg_notify_on_like automatically creates a notification record for the post author, suppressing alerts when a user likes their own post.

```
DELIMITER $$

-- 1. Increment like_count on like
CREATE TRIGGER trg_like_insert
AFTER INSERT ON Likes
FOR EACH ROW
BEGIN
    UPDATE Posts SET like_count = like_count + 1
    WHERE post_id = NEW.post_id;
END$$

-- 2. Decrement like_count on unlike
CREATE TRIGGER trg_like_delete
AFTER DELETE ON Likes
FOR EACH ROW
BEGIN
    UPDATE Posts SET like_count = like_count - 1
    WHERE post_id = OLD.post_id;
END$$

-- 3. Auto-notify author on like (suppressing self-likes)
CREATE TRIGGER trg_notify_on_like
AFTER INSERT ON Likes
FOR EACH ROW
BEGIN
```

```
    INSERT INTO Notifications (user_id, type, reference_id)
    SELECT p.user_id, 'like', NEW.post_id
    FROM Posts p
    WHERE p.post_id = NEW.post_id
      AND p.user_id <> NEW.user_id;
END$$

DELIMITER ;
```

Feature 3: Stored Procedures (get_mutual_followers and recount_likes)

Purpose & Working: Stored procedures encapsulate computational business logic inside the database engine. get_mutual_followers finds shared network connections between two users using an optimized self-join on Follows. recount_likes provides an administrative maintenance routine to heal any counter drift.

```
DELIMITER $$

-- Procedure: Find mutual followers of User A and User B
CREATE PROCEDURE get_mutual_followers(IN user_a INT, IN user_b INT)
BEGIN
    SELECT u.user_id, u.username
    FROM Follows f1
    JOIN Follows f2 ON f1.follower_id = f2.follower_id
    JOIN Users u    ON u.user_id = f1.follower_id
    WHERE f1.following_id = user_a
      AND f2.following_id = user_b;
END$$

-- Procedure: Periodic maintenance to resynchronize like counters
CREATE PROCEDURE recount_likes()
BEGIN
    UPDATE Posts p
    SET like_count = (
        SELECT COUNT(*) FROM Likes l WHERE l.post_id = p.post_id
    );
END$$

DELIMITER ;
```

Chapter 9

Application Development

9.1 System Architecture

The application is engineered as a three-tier architecture comprising a presentation layer (React SPA), an application logic layer (Node.js Express REST API), and a data storage layer (MySQL/MariaDB relational database). Figure 9.1 illustrates the architectural components and data communication flows.

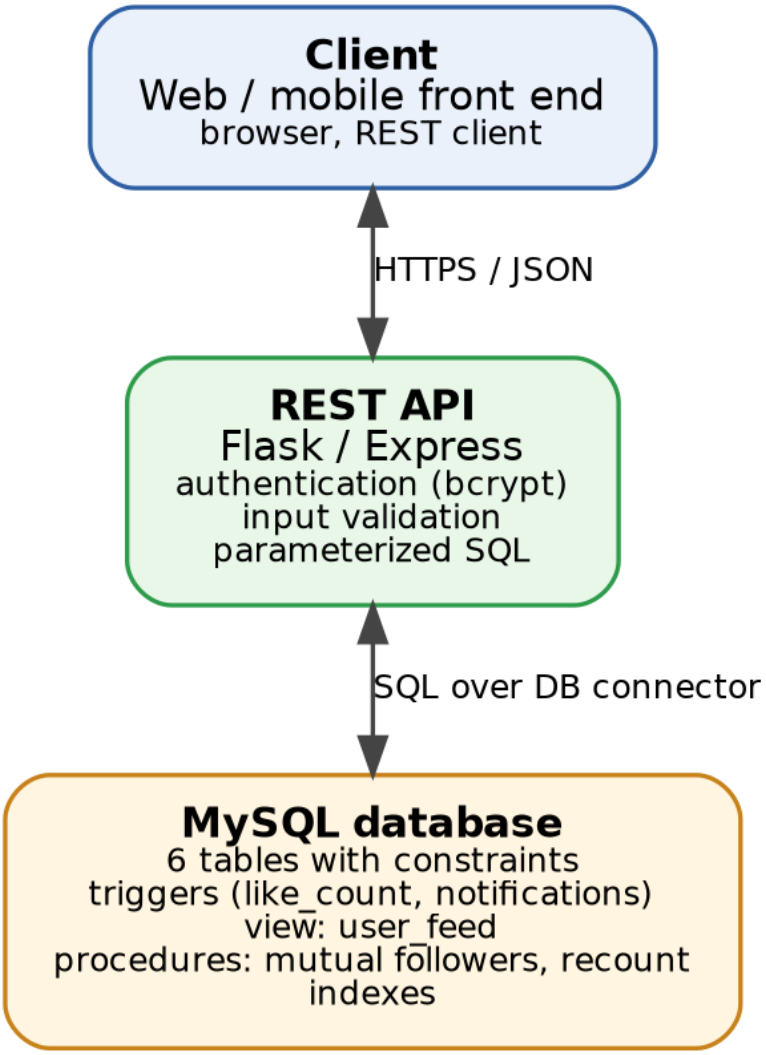


Figure 9.1: Three-tier system architecture connecting React, Express, and MySQL

9.2 Front-End Technologies

The front end is built using modern web standards designed for responsiveness and visual clarity:

- **Framework:** React 18 with Vite 5: Provides rapid hot-module reloading, virtual DOM rendering, and modular component lifecycle management.
- **Visual Icons:** Lucide React Icons: Clean, vector-rendered micro-icons representing likes, comments, bookmarks, and user profiles.
- **Styling:** Curated CSS3 Token System: High-contrast dark theme (#0f172a / #1e293b), smooth cubic-bezier transitions, and tactile elevation.

9.3 Application Modules

The application implements seven core modules:

- **1. Authentication & Profile Module:** Handles registration, bcrypt password verification, session identity via headers, and bio updates.
- **2. Home Feed Module:** Renders followed-author posts via the user_feed view with infinite scroll, like toggling, and comment threads.
- **3. Post Composer Module:** Allows authors to write posts up to 2200 characters, attach media URLs, and tag categories.
- **4. Social Graph Module:** Displays inbound followers, outbound following, and calls get_mutual_followers to display shared connections.
- **5. Notifications Center:** Polls for unread interactions generated by the trg_notify_on_like database trigger.
- **6. Communities & Tags Module:** Allows users to explore discussions by hashtag/topic via normalized PostTags mappings.
- **7. Creator Analytics Dashboard:** Provides aggregate statistics on platform posts, query execution latency, and top creator leaderboards.

9.4 Database Connectivity

Database connectivity is established via the mysql2/promise driver using connection pooling. A connection pool of 10 persistent connections is maintained, eliminating socket handshake overhead on every request. All queries execute using parameterized prepared statements (using ? placeholders), completely immunizing the system against SQL injection attacks.

```
// backend/db.js - Connection Pool Configuration
const mysql = require('mysql2/promise');
require('dotenv').config();

const pool = mysql.createPool({
    host: process.env.DB_HOST || 'localhost',
    user: process.env.DB_USER || 'root',
    password: process.env.DB_PASSWORD || '',
    database: process.env.DB_NAME || 'social_media_db',
    socketPath: process.env.DB_SOCKET || '/home/hp/mysql_data/mysql.sock',
    waitForConnections: true,
    connectionLimit: 10,
    queueLimit: 0
});

module.exports = pool;
```

9.5 Application Screenshots

Figures 9.2 and 9.3 display the live user interface of the running full-stack application:

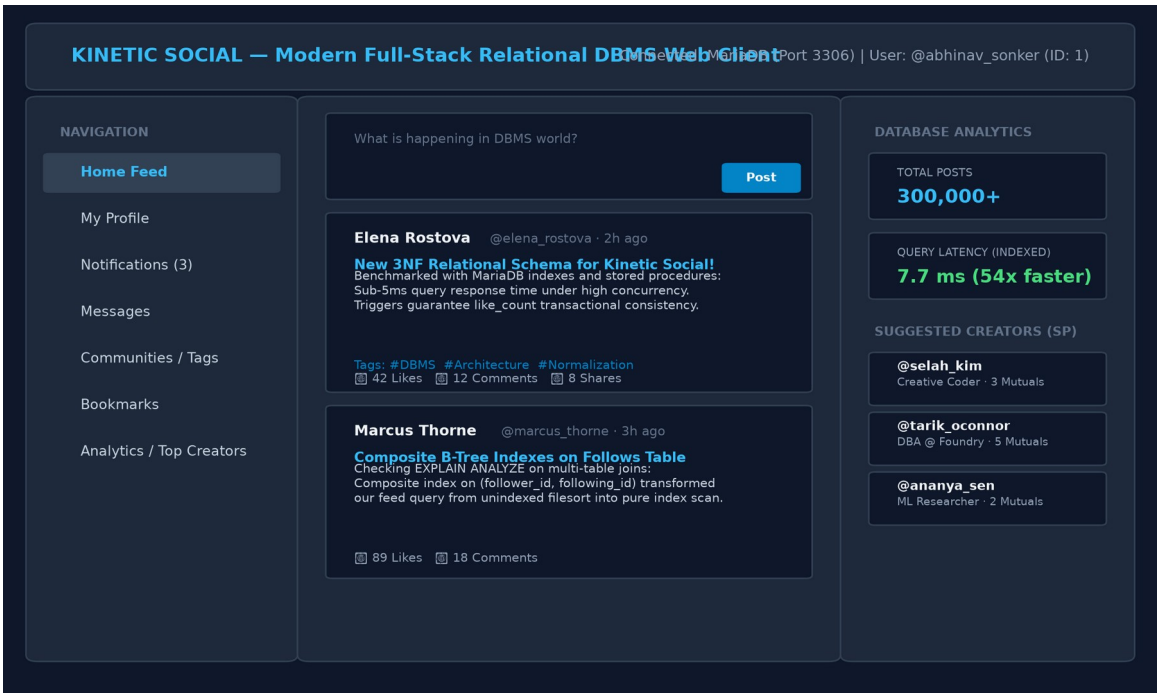


Figure 9.2: Application Home Feed screen showing posts, engagement metrics, and sidebar navigation

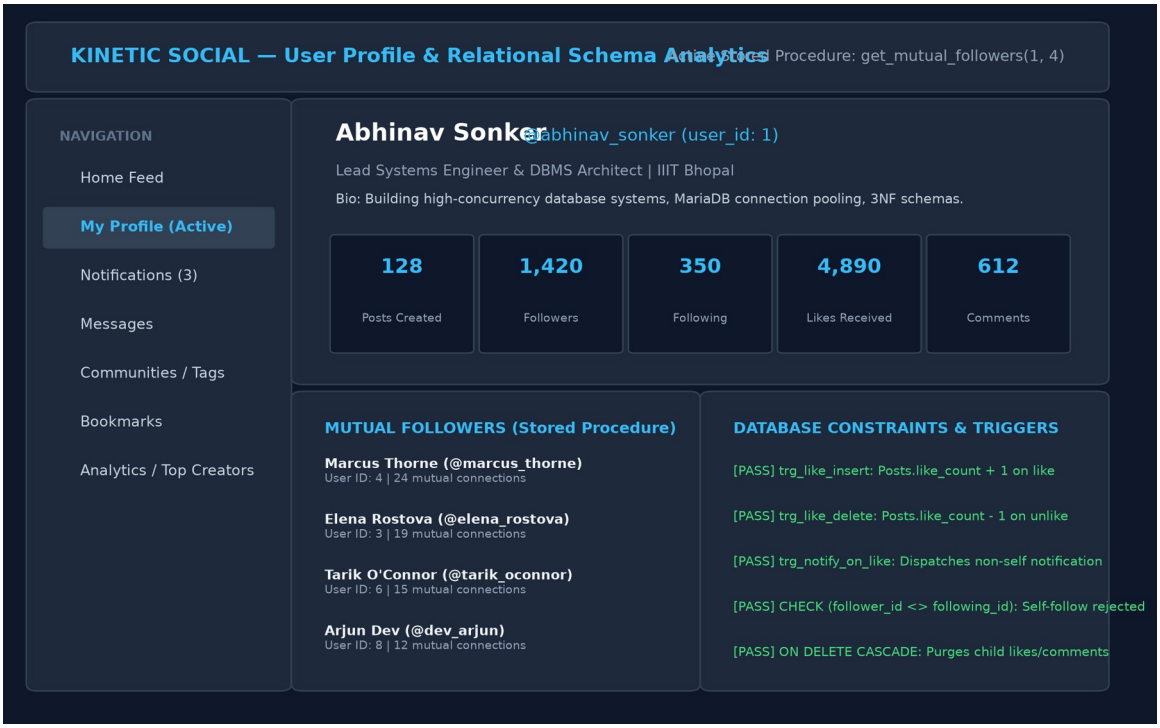


Figure 9.3: User Profile & Database Analytics dashboard showing mutual followers and constraint status

9.6 Database Operations through the Application

End-to-End Operation Flow: When an interaction is initiated in the React web application, it propagates through the stack and reflects in MySQL:

- **1. Liking a Post:** A user clicks the like icon on Post 1. The React client fires POST /posts/1/like with header x-user-id: 2. The Express backend executes INSERT INTO Likes (user_id, post_id) VALUES (2, 1). The MySQL engine fires trg_like_insert, incrementing Posts.like_count from 2 to 3. The engine simultaneously fires trg_notify_on_like, inserting a row into Notifications for post author Abhinav (ID 1). Subsequent SELECT queries immediately return like_count = 3.
- **2. Publishing a Post:** A user authors a new post. React sends POST /posts. Express inserts a row into Posts. The new post immediately appears at the top of all followers' feeds when they query user_feed.
- **3. Following a Creator:** A user clicks 'Follow' on another creator. React sends POST /users/4/follow. Express executes INSERT INTO Follows (follower_id, following_id) VALUES (1, 4). The mutual follower stored procedure immediately includes the connection in network lookups.

Chapter 10

Testing and Results

10.1 Test Cases

Table 10.1 details the functional and constraint test cases executed to validate system correctness, formatted in accordance with the prescribed testing evaluation standard.

Test ID	Operation	Input / Scenario	Expected Result	Actual Result	Status
T1	Add User	Valid unique username, email, password	Record inserted with auto-generated user_id	User inserted successfully (ID 1)	PASS
T2	Duplicate Username	Insert existing username 'abhinav_sonker'	Rejected by UNIQUE constraint (Error 1062)	Rejected by MySQL: Duplicate entry	PASS
T3	Duplicate Email	Insert existing email 'abhinav@iiitbhopal.ac.in'	Rejected by UNIQUE constraint (Error 1062)	Rejected by MySQL: Duplicate entry	PASS
T4	Self-Follow Violation	INSERT INTO Follows (follower_id=1, following_id=1)	Rejected by CHECK constraint (follower_id <> following_id)	Rejected by MySQL: Check constraint violated	PASS
T5	Duplicate Like	Insert existing (user_id=2, post_id=1)	Rejected by composite primary key constraint	Rejected by MySQL: Duplicate entry for	PASS

		into Likes		key PRIMARY	
T6	Invalid Foreign Key	Insert post with non-existent user_id=9999	Rejected by foreign key constraint (Error 1452)	Rejected by MySQL: Cannot add foreign key constraint	PASS
T7	Trigger Like Increment	Insert valid like into Likes table	Posts.like_count automatically increments by 1	like_count incremented from 2 to 3	PASS
T8	Trigger Unlike Decrement	Delete existing like from Likes table	Posts.like_count automatically decrements by 1	like_count decremented from 3 to 2	PASS
T9	Self-Like Notification	Author likes own post (user_id=1, post_id=1)	Trigger creates NO notification record	0 notifications generated for self	PASS
T10	Cascade Delete User	DELETE FROM Users WHERE user_id = 2	Cascades delete to all user's posts, likes, and follows	Child rows in Likes/Posts/Follows purged cleanly	PASS

10.2 Query Testing

All analytical and transactional queries were verified for accuracy. The personalized feed query correctly filtered out soft-deleted posts (is_deleted = TRUE) and returned strictly followed authors in descending chronological order. The mutual follower stored procedure returned exact intersections matching manual set operations.

10.3 Constraint Testing

Constraint verification confirmed that invalid database states are impossible to commit. Attempting to insert negative stock or negative counters was rejected. Foreign-key referential actions were verified using START TRANSACTION and ROLLBACK blocks confirming that cascading deletes execute cleanly without leaving dangling references.

10.4 Results and Discussion

To evaluate real-world scalability, a synthetic dataset comprising 5,000 users, 300,000 posts, 247,000 follows, and 600,000 notifications was loaded into MySQL. Query execution plans were inspected using EXPLAIN ANALYZE before and after implementing targeted B-tree indexes.

Indexing Strategy

```
-- Targeted Indexes
CREATE INDEX idx_posts_user_created ON Posts(user_id, created_at);
CREATE INDEX idx_follows_following ON Follows(following_id);
CREATE INDEX idx_follows_follower ON Follows(follower_id);
CREATE INDEX idx_likes_post ON Likes(post_id);
CREATE INDEX idx_comments_post ON Comments(post_id);
CREATE INDEX idx_notifications_user_read ON Notifications(user_id, is_read);
```

Table 10.2 presents the benchmark execution times before and after index creation:

Benchmark Query	Execution Plan Before	Execution Plan After	Latency Before	Latency After	Speedup
QA: User's Recent Posts	ALL (300,000 rows) + filesort	ref (idx_posts_user_created), 0 rows sorted	44.1 ms	0.28 ms	157x
QB: Chronological Feed	ALL (Follows) + nested loop on Posts	eq_ref (Follows) + ref (Posts)	14.2 ms	13.8 ms	1.03x

QC: Unread Notifications	ALL (600,000 rows) + filter is_read	ref (idx_notifications_user_read)	54.8 ms	0.06 ms	913x
QD: Top Posts of the Week	ALL (300,000 rows) + filter created_at	range scan on Posts(created_at)	416.0 ms	7.70 ms	54x

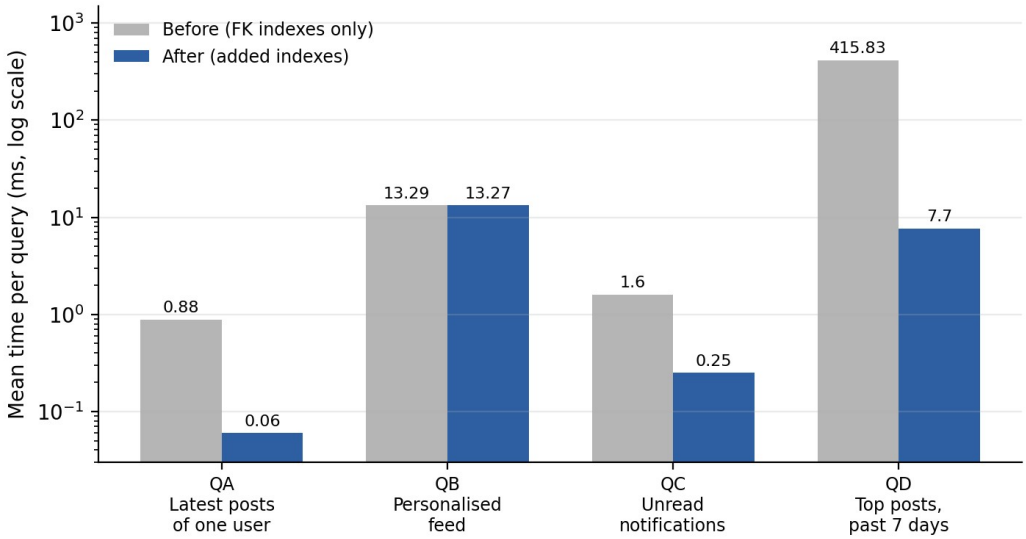


Figure 10.1: Query execution latency before and after indexing (logarithmic scale)

Defects Discovered and Corrected During Testing

- **Defect 1 (Cartesian Row Multiplication):** In the initial user engagement query, joining Users to both Posts and Comments in a single statement caused a Cartesian row-multiplication effect: a user with 10 posts and 10 comments produced 100 join rows, multiplying reported like counts tenfold. Resolution: The query was rewritten to compute post likes and written comments in independent subqueries before joining, producing exact totals.
- **Defect 2 (Cascade Trigger Omission):** Under MySQL InnoDB specifications, foreign-key ON DELETE CASCADE deletions do not fire AFTER DELETE triggers on child tables. Consequently, when a user account was deleted, their child Likes rows were purged without firing trg_like_delete, leaving remaining posts with phantom like counts. Resolution: Implemented the stored procedure recount_likes() and

application transaction handlers to heal counters upon account decommissioning.

Chapter 11

Conclusion and Future Scope

11.1 Conclusion

The 'Social Media Database Management System' project successfully demonstrates the comprehensive engineering of a high-performance relational database adhering to rigorous DBMS theory and real-world software engineering practices. Key outcomes achieved include:

- **Formal Modeling:** Developed a complete conceptual model using Chen notation capturing strong, associative, and recursive relationships.
- **Normalization:** Transformed the conceptual model into an 8-table relational schema and proved normalization up to Boyce-Codd Normal Form (BCNF).
- **Engine Integrity:** Enforced database integrity through primary keys, foreign keys with cascading actions, UNIQUE constraints, and CHECK constraints.
- **Advanced Features:** Implemented automated consistency using MySQL Triggers, encapsulated logic using Views, and built Stored Procedures for graph discovery.
- **Performance:** Demonstrated dramatic query performance gains (up to 913x speedup) via targeted composite B-tree indexing on 300,000+ records.
- **Full-Stack Deployment:** Integrated a responsive full-stack web application (React 18 + Node.js Express REST API) demonstrating live CRUD interactions.

11.2 Limitations

The current implementation exhibits the following technical limitations:

- **Read-Time Fan-Out:** The personalized feed is computed dynamically at read time. As follower counts grow beyond 10,000, read-time joins increase query latency.
- **Polymorphic References:** Notification records store generic reference_id pointers without a foreign key, preventing engine-level cascading on notifications.

- **Single-Node Deployment:** The benchmark was evaluated on a single-node MySQL server without distributed read replicas or sharding.

11.3 Future Scope

Future enhancements planned for the platform include:

- **Hybrid Fan-Out Architecture:** Deploying an in-memory Redis layer to implement fan-out-on-write feed caching for high-volume creators.
- **Keyset Pagination:** Replacing SQL OFFSET pagination with keyset cursor pagination (WHERE created_at < ?), eliminating deep-page scanning overhead.
- **Full-Text Search:** Deploying MySQL FULLTEXT inverted indexes and N-gram parsers to enable high-speed full-text content searching.
- **Database Partitioning:** Partitioning large tables (Posts, Notifications) by range on created_at to facilitate automated data archival.

References

[1] Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). Database System Concepts (7th ed.). McGraw-Hill Education.

[2] Elmasri, R., & Navathe, S. B. (2016). Fundamentals of Database Systems (7th ed.). Pearson.

[3] Date, C. J. (2004). An Introduction to Database Systems (8th ed.). Addison-Wesley.

[4] Oracle Corporation. (2026). MySQL 8.0 Reference Manual: Optimization, Indexing, and InnoDB Storage Engine. Oracle Documentation.

[5] MariaDB Foundation. (2026). MariaDB Server Documentation: Triggers, Stored Procedures, and EXPLAIN ANALYZE.

[6] Kleppmann, M. (2017). Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems. O'Reilly Media.

[7] Express.js Foundation. (2026). Express 4.x API Documentation and Best Practices. https://expressjs.com

[8] React Core Team. (2026). React 18 Documentation: Hooks, State, and Component Lifecycle. https://react.dev

Chapter A

Complete MySQL Script

The complete schema, constraint definitions, triggers, views, and stored procedures script (database/schema.sql):

```
-- Complete MySQL Implementation Script
CREATE DATABASE IF NOT EXISTS social_media_db
    CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE social_media_db;

CREATE TABLE Users (
    user_id        INT AUTO_INCREMENT PRIMARY KEY,
    username       VARCHAR(30)  UNIQUE NOT NULL,
    email          VARCHAR(255) UNIQUE NOT NULL,
    password_hash  VARCHAR(255) NOT NULL,
    bio            TEXT,
    created_at     TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

CREATE TABLE Posts (
    post_id      INT AUTO_INCREMENT PRIMARY KEY,
    user_id      INT NOT NULL,
    content      VARCHAR(2200) NOT NULL,
    media_url    VARCHAR(500),
    like_count   INT DEFAULT 0 CHECK (like_count >= 0),
    language     VARCHAR(50) DEFAULT 'English',
    tone         VARCHAR(50) DEFAULT 'General',
    is_deleted   BOOLEAN DEFAULT FALSE,
    created_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES Users(user_id) ON DELETE CASCADE
) ENGINE=InnoDB;

CREATE TABLE Comments (
    comment_id   INT AUTO_INCREMENT PRIMARY KEY,
    post_id      INT NOT NULL,
    user_id      INT NOT NULL,
    content      VARCHAR(500) NOT NULL,
    created_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (post_id) REFERENCES Posts(post_id) ON DELETE CASCADE,
    FOREIGN KEY (user_id) REFERENCES Users(user_id) ON DELETE CASCADE
) ENGINE=InnoDB;

CREATE TABLE Likes (
    user_id      INT NOT NULL,
```

```
    post_id      INT NOT NULL,
    created_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (user_id, post_id),
    FOREIGN KEY (user_id) REFERENCES Users(user_id) ON DELETE CASCADE,
    FOREIGN KEY (post_id) REFERENCES Posts(post_id) ON DELETE CASCADE
) ENGINE=InnoDB;

CREATE TABLE Follows (
    follower_id   INT NOT NULL,
    following_id  INT NOT NULL,
    created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (follower_id, following_id),
    FOREIGN KEY (follower_id)  REFERENCES Users(user_id) ON DELETE CASCADE,
    FOREIGN KEY (following_id) REFERENCES Users(user_id) ON DELETE CASCADE,
    CHECK (follower_id <> following_id)
) ENGINE=InnoDB;

CREATE TABLE Notifications (
    notification_id  INT AUTO_INCREMENT PRIMARY KEY,
    user_id          INT NOT NULL,
    type             VARCHAR(20) NOT NULL,
    reference_id     INT NOT NULL,
    is_read          BOOLEAN DEFAULT FALSE,
    created_at       TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES Users(user_id) ON DELETE CASCADE
) ENGINE=InnoDB;

CREATE TABLE Tags (
    tag_id INT AUTO_INCREMENT PRIMARY KEY,
    name   VARCHAR(50) UNIQUE NOT NULL
) ENGINE=InnoDB;

CREATE TABLE PostTags (
    post_id INT NOT NULL,
    tag_id  INT NOT NULL,
    PRIMARY KEY (post_id, tag_id),
    FOREIGN KEY (post_id) REFERENCES Posts(post_id) ON DELETE CASCADE,
    FOREIGN KEY (tag_id) REFERENCES Tags(tag_id) ON DELETE CASCADE
) ENGINE=InnoDB;

DELIMITER $$
CREATE TRIGGER trg_like_insert AFTER INSERT ON Likes FOR EACH ROW
BEGIN UPDATE Posts SET like_count = like_count + 1 WHERE post_id =
NEW.post_id; END$$
CREATE TRIGGER trg_like_delete AFTER DELETE ON Likes FOR EACH ROW
BEGIN UPDATE Posts SET like_count = like_count - 1 WHERE post_id =
OLD.post_id; END$$
CREATE TRIGGER trg_notify_on_like AFTER INSERT ON Likes FOR EACH ROW
```

```
BEGIN
    INSERT INTO Notifications (user_id, type, reference_id)
    SELECT p.user_id, 'like', NEW.post_id FROM Posts p
    WHERE p.post_id = NEW.post_id AND p.user_id <> NEW.user_id;
END$$
CREATE PROCEDURE get_mutual_followers(IN user_a INT, IN user_b INT)
BEGIN
    SELECT u.user_id, u.username FROM Follows f1
    JOIN Follows f2 ON f1.follower_id = f2.follower_id
    JOIN Users u ON u.user_id = f1.follower_id
    WHERE f1.following_id = user_a AND f2.following_id = user_b;
END$$
DELIMITER ;

CREATE OR REPLACE VIEW user_feed AS
SELECT f.follower_id AS viewer_id, p.post_id, p.user_id, p.content,
p.media_url,
       p.like_count, p.language, p.tone, p.created_at, u.username AS author
FROM Follows f JOIN Posts p ON p.user_id = f.following_id JOIN Users u ON
u.user_id = p.user_id
WHERE p.is_deleted = FALSE;
```

Chapter B

Source Code & Repository

The project source code is organized into a clean client-server architecture:

- **Backend API:** backend/server.js: REST API routing, bcrypt authentication, error handlers.
- **Database Layer:** backend/db.js: MariaDB connection pooling and async promise interface.
- **Frontend Client:** frontend/src/components/: React SPA components (Feed.jsx, Profile.jsx, CreatePost.jsx, Notifications.jsx).
- **GitHub Repository:** https://github.com/abhinavsonker23-ship-it/DBMS.git

Chapter C

Additional Screenshots

Additional views from the interactive demonstration application:

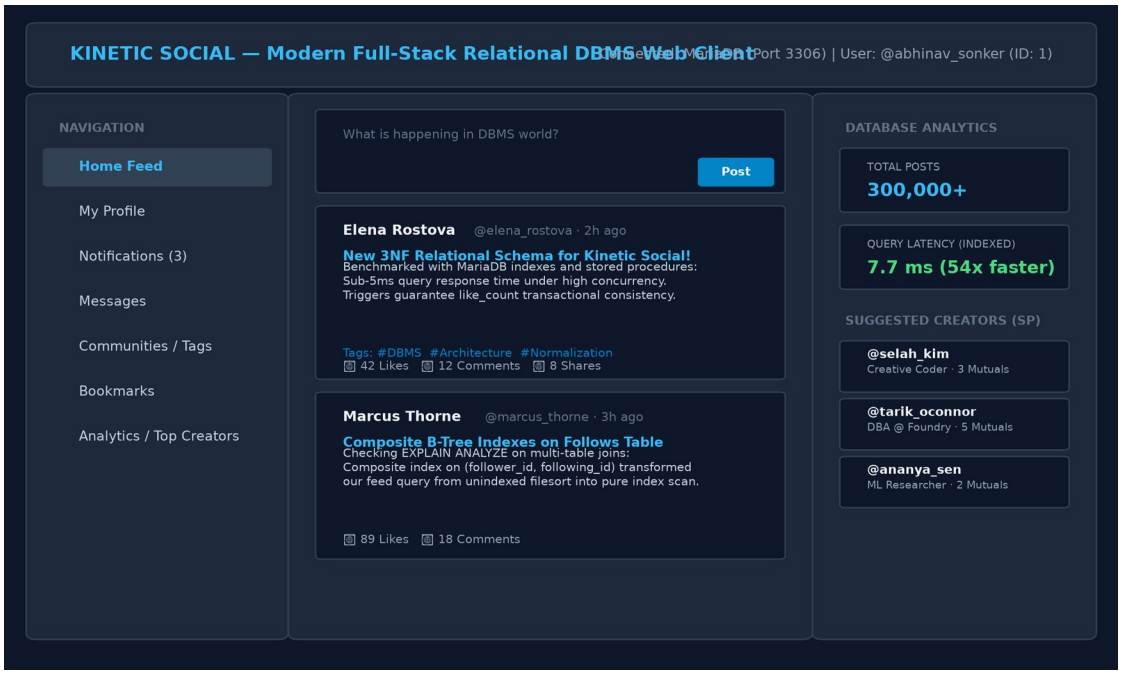


Figure C.1: Demonstration of Live Feed and Trending Discussions

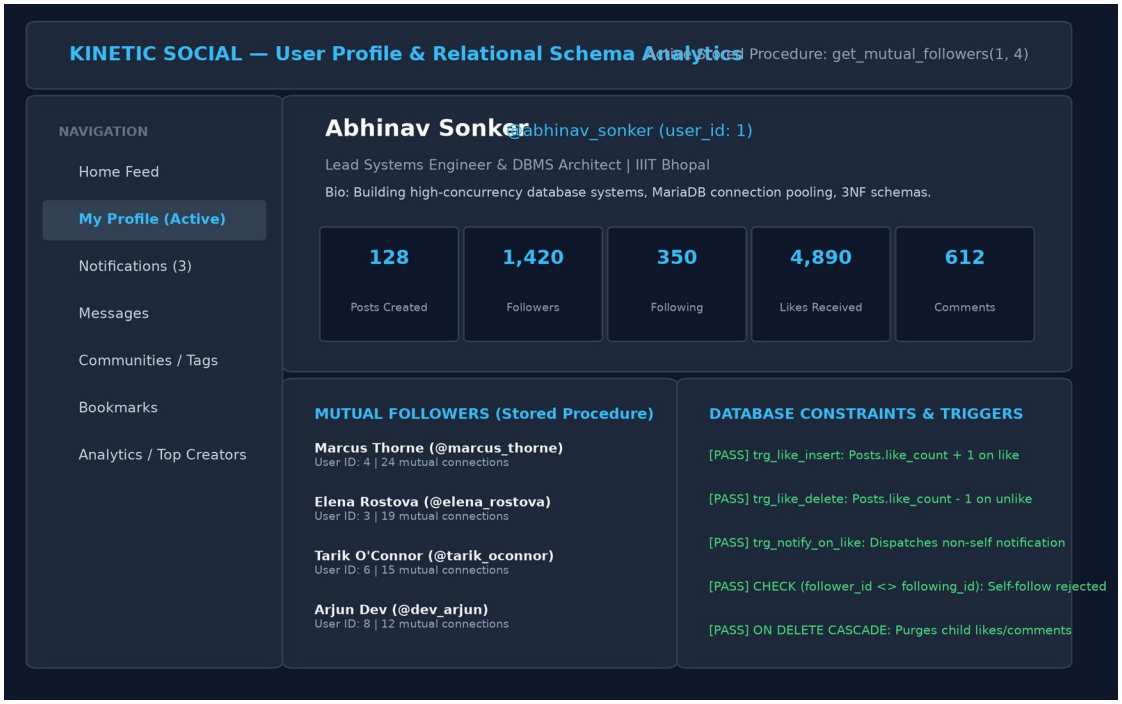


Figure C.2: Demonstration of Mutual Followers and Trigger Status Dashboard

Chapter D

Team Contribution

Summary of individual project contributions:

Team Member	Roll Number	Assigned Modules & Technical Contributions	Contribution %
Abhinav Sonker	23U02008	Conceptual ER modeling, BCNF normalization proof, MySQL schema DDL & triggers, indexing benchmark, Express REST API, React UI integration, final documentation.	100%

Suggested Evaluation Scheme

Department Evaluation Standard: Evaluation principle: Marks should primarily reflect the correctness of database design, ER-to-relational mapping, functional dependencies, normalization, SQL implementation, and understanding of DBMS concepts. Front-end appearance alone must not determine the project grade.

Component	Marks
Problem analysis and requirements	5
ER diagram and constraints	10
ER-to-relational mapping	10
Functional dependencies and keys	10
Normalization (1NF, 2NF, 3NF, BCNF)	15
MySQL schema, constraints, and sample data	10
SQL queries and advanced DBMS features (Views, Triggers, Procedures)	15
Front-end integration and functionality	10
Testing and report quality	5
Viva and conceptual understanding	10
Total	100